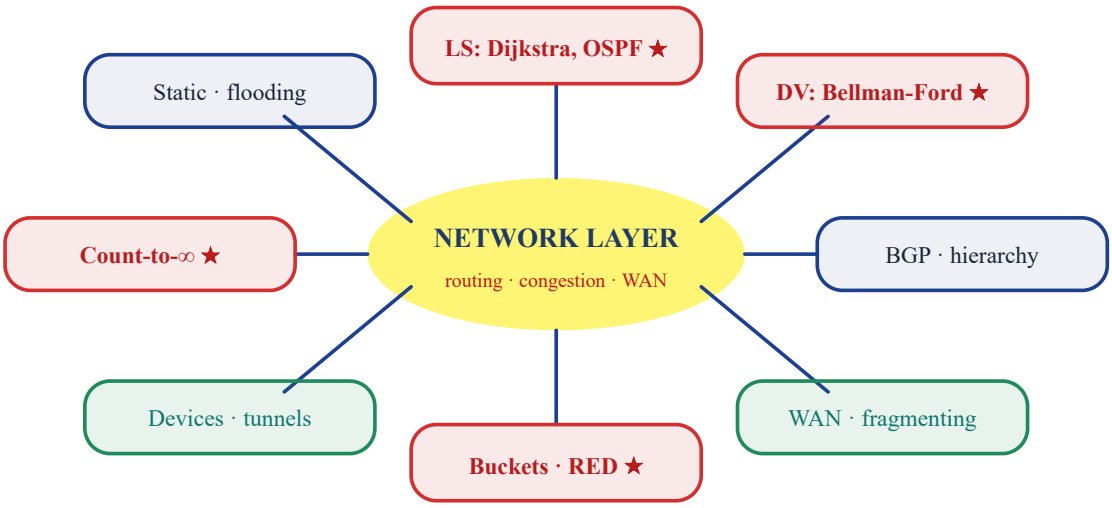
Routing, Congestion Control & WAN

Dijkstra · Bellman-Ford · BGP · buckets · devices · X.25/FR/ATM/MPLS · fragmentation: full exam prep in 12 sheets



↑ the whole lesson on one page: red = high-yield, green = supporting

The syllabus line

"Computer Networking and Security: … **Routing algorithm**; **Congestion control**; **Internetworking, WAN**; **Fragmentation**; …". The sibling notes cover: layers and LAN (IEEE 802), IPv4/IPv6 header and addressing, ARP (network layer note), TCP's congestion window (transport layer note).

High-yield list

① DV → Bellman-Ford
② LS → Dijkstra
③ RIP: 16 hops = ∞
④ Split horizon
⑤ $S = C/(M - \rho)$
⑥ offset ÷ 8 bytes
⑦ ATM cell 53 B

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

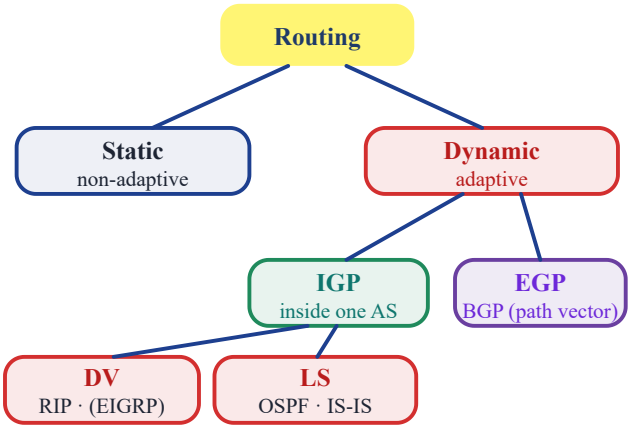
Contents

✓ Routing basics, flooding …… p. 2
✓ Link state, Dijkstra, OSPF, hierarchy … p. 3
✓ Distance vector, RIP, count-to-∞ … p. 4
✓ BGP, broadcast, congestion …… p. 5
✓ Choke packets, RED, shaping …… p. 6
✓ Leaky vs token bucket, devices … p. 7
✓ Tunnelling, switching …… p. 8
✓ WAN: leased lines, X.25, FR, ATM, MPLS … p. 9
✓ IPv4 fragmentation …… p. 10
✓ Night-before sheet …… p. 11
✓ Practice MCQ + written …… p. 12

1 Routing: the Whole Picture

Definition: A **routing algorithm** is the network-layer (layer 3) software that decides which output line an arriving packet takes. **Routing** = building the routing table (control plane); **forwarding** = consulting that table for each packet (data plane).

Optimality principle: If J lies on an optimal path from I to K, then the optimal path from J to K follows that same route. So all the best routes towards one destination together form a tree: a **sink tree** (no loops).



the routing family tree

	Static (non-adaptive)	Dynamic (adaptive)
Route set by	the administrator, by hand	routers running protocols
Failure · overhead · use	no reaction · none · small nets	self-correcting · bandwidth, CPU · big nets

Metric: hop count (RIP) · bandwidth · delay · load · reliability · MTU · cost (OSPF) · AS path / policy (BGP). **The lower the metric, the better the route**.

Memory trick: "**RIP counts hops, OSPF counts cost, BGP counts ASes** (and that means policy)".

Exam trap: The **metric** compares routes **inside one protocol**; the **administrative distance** (AD, how trusted a source is) chooses **between different protocols**, and the lower value wins. Cisco AD: connected 0 · static **1** · eBGP 20 · EIGRP 90 · OSPF 110 · IS-IS 115 · RIP 120 · iBGP 200. So whatever the hop count, an OSPF route beats a RIP route.

2 Flooding

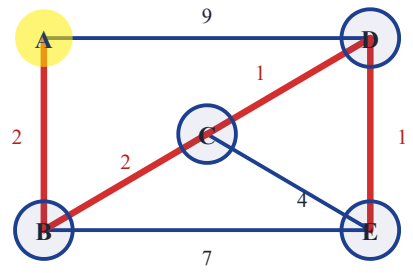
A static algorithm: every packet is sent on **every line except the one it came in on**. It is robust and always finds the shortest path (every path is tried), but the network fills with duplicate packets. Control it with a **hop counter** (starting at the network radius), **sequence numbers** (drop copies already seen) or **selective flooding**. Flooding is what spreads link-state packets.

3 Link-State Routing and Dijkstra

Core idea: Every router knows a **map of the whole topology** and runs the **Dijkstra algorithm** (SPF) over it: "routing by map".

1. **Find the neighbours** (HELLO) · **measure every cost** · **build the link-state packet** (LSP/LSA: ID, sequence no., age, neighbours + costs)
2. Flood the LSP to **every** router · run **Dijkstra over the whole map** → shortest-path tree → routing table

Worked example: shortest path from A



red = the final shortest-path tree

Step	Permanent set	B	C	D	E
0	{A}	**2,A**	∞	9,A	∞
1	{A,B}	✓	**4,B**	9,A	9,B
2	{A,B,C}	✓	✓	**5,C**	8,C
3	{A,B,C,D}	✓	✓	✓	**6,D**

"4,B" means distance 4, previous node B. At every step the **smallest tentative** node becomes permanent and its neighbours are relaxed. The next hop from A to every destination is **B**; A→E = A-B-C-D-E, cost 6 (not the obvious A-D-E = 10).

OSPF facts:

RFC 2328 (v2), v3 for IPv6

runs **straight over IP**: protocol **89**

cost = 10^8 / bandwidth (100 Mbps → 1)

areas; backbone = **area 0**; ABR, ASBR, DR/BDR

224.0.0.5 all routers · 224.0.0.6 DR

Hello 10 s, dead 40 s · AD 110

Memory trick: OSPF in numbers: "**89 · 110 · area 0 · 5/6 · 10/40**" = protocol, AD, backbone, last digits of the multicast addresses, hello/dead.

IS-IS: the OSI link-state twin — not IP, it runs straight on layer 2. Dijkstra: $O(N^2)$.

Exam trap: **Negative** edge costs make Dijkstra return a wrong answer; Bellman-Ford copes (as long as there is no negative cycle).

4 Hierarchical Routing

Routers are grouped into **regions** (areas, ASes); a router knows every router in its own region but keeps only **a single entry** for each other region. Tables stay small, and paths can be slightly longer. Optimal: **ln N** levels, **e·ln N** entries.

Worked example ✍️: 720 routers. Flat: **720** entries. 24 regions × 30 routers: 30 + 23 = **53**. 8 clusters × 9 regions × 10 routers: 10 + 8 + 7 = **25**.

5 Distance-Vector Routing and Bellman-Ford

Core idea: Every router keeps the best **distance** (and line) it knows to each destination, and exchanges this vector periodically with **its neighbours only**. "Routing by rumour"; it ran ARPANET until 1979, and it runs RIP today.

Bellman-Ford equation: $D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$ over every neighbour v of x

Worked example ✍️: Router X: links X–A = 2, X–B = 5, X–C = 1. Vectors received:

From	To Y	To Z
A	7	3
B	2	9
C	8	6

Y: min(2+7, **5+2**, 1+8) = **7, via B**

Z: min(**2+3**, 5+9, 1+6) = **5, via A**

RIP facts:

metric = **hop count**; maximum **15**, **16 = ∞**

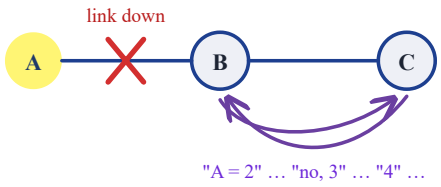
whole table every **30 s** · UDP port **520**

invalid 180 s · hold-down 180 s · flush 240 s

v1 classful, broadcast · v2 classless, multicast **224.0.0.9**, authentication

RIPng (IPv6): UDP 521 · AD 120

6 Count-to-Infinity and Its Defences



Exchange	B's dist to A	C's dist to A
Before failure	1	2
1	3 (via C!)	2
2 · 3 · 4	3 · 5 · 5	4 · 4 · 6
Finally	**16 = ∞**	**16 = ∞**

C keeps advertising "A is 2 away" (that route really runs back through B), and B believes it. **Good news travels fast, bad news travels slowly**.

Defence	How it works
Split horizon · poison reverse	a route is not advertised back out the interface it was learned on · or it is sent back with metric 16 (∞)
Route poisoning · triggered update	a failed route is sent at once with metric 16 instead of waiting 30 s
Hold-down timer	once a route is down, bad news about it is ignored for 180 s

Exam trap: Split horizon cures a loop between **two** routers, but not a loop of **three or more**; in RIP it only bounds the small infinity, metric 16.

	Distance vector	Link state
Knows · algorithm	neighbour distances · Bellman-Ford	full topology · Dijkstra
Sends	whole table, to neighbours, periodically	its own LSA, to everyone, on change
Convergence · CPU	slow, count-to-∞ · low, small nets	fast, loop-free · high, big nets
Examples	RIP, IGRP; EIGRP = "hybrid DV" (DUAL)	OSPF, IS-IS

7 BGP: Routing Between Autonomous Systems

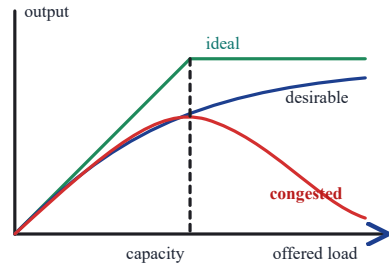
- A **path vector** protocol; BGP-4 is the Internet's inter-domain (EGP) protocol
- runs on **TCP port 179**; eBGP between ASes, iBGP inside one AS
- every route carries its whole **AS_PATH**; a router that finds its own AS in the path rejects it → **no loops**
- it chooses by **policy** (LOCAL_PREF, AS_PATH length, MED), not by cost · messages: OPEN, UPDATE, KEEPALIVE, NOTIFICATION

8 Broadcast and Multicast Routing

Broadcast: a separate copy per host (wasteful) · flooding · **multidestination** · a **spanning tree** (best) · **reverse path forwarding** (RPF): forward the copy only if it arrived on the very interface this router would use to reach the source itself. **Multicast** (224.0.0.0/4, IGMP): either a source-based tree (DVMRP, PIM-DM) or a shared core-based tree (CBT, PIM-SM).

9 Congestion and Its Control

Definition: **Congestion**: too many packets in the network (or in some part of it), so delay rises and throughput falls. **Congestion control** is a whole-network matter (global); **flow control** is point-to-point (a fast sender, a slow receiver).



offered packets vs delivered packets

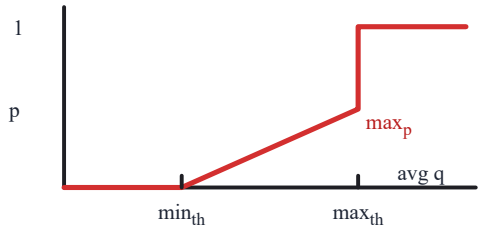
Causes: bursty traffic from several lines piling onto one line, slow routers, small buffers, low-bandwidth lines.

Infinite buffers make it **worse**: packets are delayed until they time out and are retransmitted, so the load rises further. Once capacity is passed the output can drop almost to zero: **congestion collapse**.

	Open loop (prevention)	Closed loop (feedback, reaction)
Idea	good design, so congestion never starts	measure, signal, then adjust
Means	provisioning, **admission control**, traffic shaping (buckets), retransmission/ACK/discard policy	**choke packets**, back-pressure, ECN, RED, TCP's AIMD window (transport note)

- **Admission control**: on a virtual-circuit network a new VC is refused when it would congest the network (like the "busy" tone of a telephone).
- **Choke packet**: the congested router sends a packet **back to the source**, and the source cuts its rate (say, in half). **Hop-by-hop back-pressure**: the choke packet is effective at every router along the path, so relief is immediate. (ICMP Source Quench is now obsolete.)
- **ECN**: the router drops nothing; it **marks** 2 bits in the IP header, and the receiver reports it back (TCP ECE flag), so the sender slows down.
- **Load shedding**: throwing packets away. **Win** (the older one is better: keep file transfer, drop the newcomer) against **milk** (the newer one is better: real-time media, drop the old).

RED (Random Early Detection, Floyd & Jacobson 1993): the router watches the **average** queue length. Below $\min_{th}$: nothing is dropped; between the two thresholds: drop with a probability rising to $\max_p$; above $\max_{th}$: drop everything. Dropping **early and at random** slows only a few TCP senders and avoids the **global synchronisation** of tail drop.

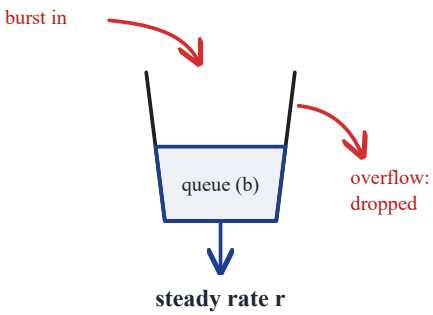


RED's drop probability

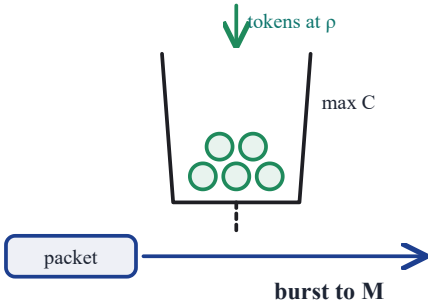
Exam trap: Congestion control ≠ flow control: sending fast from a PC to a slow printer over an **empty** link is a flow-control problem; many hosts on one overloaded link is congestion.

10 Traffic Shaping: Leaky and Token Buckets

Traffic shaping: limiting the **rate and burstiness** of the traffic a host sends, according to an agreement with the network (open loop). **Policing** = the network checks it and drops or marks whatever exceeds the agreement.



leaky bucket: smooth output



token bucket: bursts pass

	Leaky bucket	Token bucket
Output	however bursty the input, a **steady** rate r	average rate ρ, bursts up to M
Idle host · when full	gets nothing · drops the **packet**	saves tokens up to C · drops the **token**
Max data in t	r·t	**C + ρt** (and ≤ M·t)

Burst length: Full bucket C bytes, token rate ρ, maximum output rate M. A burst of S seconds: C + ρS = MS ⇒ **S = C / (M − ρ)**

Example 1 ✍️: C = 250 KB, M = 25 MB/s, ρ = 2 MB/s: S = 250 KB / 23 MB/s ≈ **11 ms** at full rate, then 2 MB/s.

Example 2 ✍️: A 1 MB burst (40 ms at 25 MB/s) through a leaky bucket with output 2 MB/s: the output takes **500 ms**; the bucket must hold ≈ 0.92 MB.

Memory trick: **Leaky** = a dripping tap: always the same drop. **Token** = a savings account: save when idle, spend in a burst, but never more than C.

11 Internetworking and Its Devices

Definition: **Internetworking** = running networks of different kinds (addressing, packet size, services, QoS) as if they were one network; it is the layer-3 IP routers that hold the Internet together.

Device	Layer	Works on	Collision domain	Broadcast domain
Repeater	1	bits: regenerate signal	1 (it extends)	1
Hub	1	bits: multiport repeater	1 over all ports	1
Bridge	2	frames: filter by MAC	1 per port	1
Switch	2	frames: multiport bridge	**1 per port**	1 (per VLAN)
Router	3	packets: IP address	1 per port	**1 per port**
Gateway	4–7	protocol translation	–	–

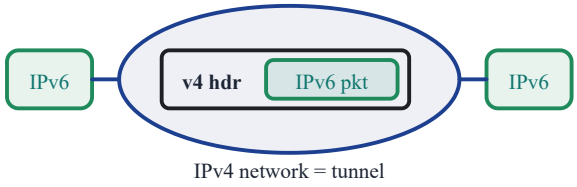
- **Bridge**: learns which port leads to a MAC from the source address (**backward learning**); floods frames for an unknown destination; STP (802.1D) removes loops.
- **Switch**: store-and-forward (verifies the CRC) or cut-through (lower latency); full duplex, no collisions; a layer-3 switch routes as well.
- **Gateway**: converts between different protocols (e-mail, VoIP); the TCP/IP "default gateway" is really just a router.

> **Worked example ✍️:** A router with two interfaces: on one, a 24-port switch with 20 PCs; on the other, a hub with 6 PCs. Collision domains: 20 + 1 (the switch–router link) + 1 (the whole hub) = **22**. Broadcast domains: **2** (one per router interface).

> **Exam trap:** A **switch** divides the collision domain but **not** the broadcast domain; only a **router** (or a VLAN) divides the broadcast domain. A hub is a layer-1 device however many ports it has.

Tunnelling

To cross a network of a **different** kind, the whole packet travels between two multiprotocol routers as the **payload** of that network's own packets. Examples: IPv6-in-IPv4 (protocol 41), GRE (protocol 47), IPsec VPN.



12 Circuit vs Packet Switching

	Circuit	Datagram	Virtual circuit
Idea	a path set up in advance	every packet routed alone	path set at setup, link shared
In the packet	–	full address	small VC number
Order · router fails	in order · call cut	may reorder · its packets lost	in order · its VCs end
QoS, congestion	easy (reserved)	hard	easy if resources reserved
Examples	PSTN, ISDN	**IP**	**X.25, Frame Relay, ATM, MPLS**

> **Delay formula:** message x bits, k hops, bit rate b, propagation d per hop, setup s, packet p bits (header excluded): circuit $\mathbf{T = s + x/b + k{\cdot}d}$ · packet $\mathbf{T = x/b + (k-1){\cdot}p/b + k{\cdot}d}$. Packet switching is faster when $s > (k-1){\cdot}p/b$.
> **Example ✍️** $x = 10^6$ bits, b = 1 Mbps, k = 4, d = 5 ms, s = 200 ms, p = 10 000 bits: circuit 0.2 + 1 + 0.02 = **1.22 s**; packet 1 + 3 × 0.01 + 0.02 = **1.05 s**.

> **Exam trap:** A virtual circuit is really **packet** switching (the links are shared), not circuit switching; it only fixes the route at setup time.

13 WAN Technologies

A **WAN** spreads across cities, countries and continents, usually over carrier lines: leased lines, circuit-switched ISDN, packet-switched X.25 / Frame Relay / ATM, MPLS, Internet VPN (IPsec), satellite/cellular backup.

Leased line	Rate	Channel
T1 (USA)	**1.544 Mbps**	24 × 64 k
E1 (Europe, BD)	**2.048 Mbps**	32 (30 usable)
T3 / E3	44.736 / 34.368	672 / 480
OC-3 = STM-1	155.52 Mbps	SONET/SDH
OC-12 = STM-4	622.08 Mbps	

ISDN: BRI = 2B + D = 2 × 64 + 16 = **144 kbps**; PRI = 23B + D (T1) or 30B + D (E1).

T1 = 24 × 64 kbps + 8 kbps framing = 1.544 Mbps. OC-1 = 51.84 Mbps; OC-n = n × 51.84.

	X.25	Frame Relay	ATM
Time · body	1976, CCITT/ITU-T	early 1990s	1990s, backbone of B-ISDN
Layer	3 (X.21, LAPB, PLP)	layer 2 only (LAPF)	physical, ATM, AAL
Unit	packet (variable)	frame (variable)	**53-byte cell**
Error control	**hop-by-hop** (noisy line)	none: drop bad frames	HEC on the header only
VC identifier	LCN	**DLCI** (10 bits)	**VPI/VCI**
Speed	≤ 64 kbps (later 2 Mbps)	56 kbps – 45 Mbps	155.52 / 622.08 Mbps
Extra	PAD for terminals	CIR, DE bit, FECN/BECN	CBR, VBR, ABR, UBR



the ATM cell



an MPLS label entry

ATM: 48 bytes was the compromise (the USA wanted 64, Europe 32). UNI header: GFC 4, VPI 8, VCI 16, PT 3, CLP 1, HEC 8 bits. The ATM network layer and AAL are in their own note.

MPLS ("layer 2.5", RFC 3031): the ingress **LER** puts the packet into a FEC and **pushes** a label; the core **LSR**s just **swap** labels (an exact match, very fast); at the egress the label is **popped**. That path is an **LSP**. Carriers use it for traffic engineering, QoS and L3 VPN; it carries any protocol.

Memory trick: **X.25 checks at every hop, Frame Relay checks nowhere, ATM cuts everything into 53.** Frame Relay = "X.25 that throws away error control, because fibre is clean".

Exam trap: ATM's HEC protects only the **5-byte header**, not the payload. Frame Relay **detects** errors (FCS) but never **corrects** them: it drops the frame.

14 IPv4 Fragmentation

Why: a datagram bigger than the next link's **MTU** (the largest payload one frame can carry) is split into several **fragments**, each a complete IP datagram with its own header. In IPv4 the **source or any router** may fragment; **reassembly happens only at the final destination**.

Header field (bits)	Role
Identification (16)	same in every fragment
DF (1) · MF (1)	not a fragment · more follow
Fragment offset (**13**)	**8-byte** units
Total length (16)	header + data ≤ 65 535

Link	MTU (B)
Ethernet · PPP	**1500**
802.11 · FDDI	2304 · 4352
X.25 (usually)	576
IPv6 minimum	1280

Worked example 1 ✍️: Total length 4000 B (20-byte header), next link MTU 1500.

1. data = 4000 − 20 = 3980 B. Max data per fragment = 1500 − 20 = 1480 (a multiple of 8 ✓)
2. split: 3980 = 1480 + 1480 + 1020

Frag	Data bytes	Total length	Offset	MF
1	0 – 1479	1500	0	1
2	1480 – 2959	1500	1480/8 = 185	1
3	2960 – 3979	1040	**2960/8 = 370**	**0**

Example 2: the 8-byte trap ✍️: MTU 1000, datagram 2400 B. 1000 − 20 = 980, which is not a multiple of 8 → take **976**. Data 2380 = 976 + 976 + 428; offsets **0, 122, 244**.

Example 3: reading a fragment ✍️: offset 100, HLEN 5, total length 100 → first data byte = 100 × 8 = **800**, data = 80 B, last byte **879**.

- MF = 1, offset = 0 → **first** fragment · MF = 0, offset ≠ 0 → **last** · MF = 0, offset = 0 → not fragmented at all
- if one fragment is lost the **whole datagram** is discarded (the reassembly timer)
- **Path MTU discovery**: send with DF = 1; a router that cannot forward drops the packet and returns ICMP type 3 code 4 "fragmentation needed" together with the next hop's MTU
- **IPv6**: a router **never** fragments; only the source does (a Fragment header, 8 B, next header 44); if it is still too large → ICMPv6 "Packet Too Big"

Memory trick: "**offset × 8 = byte position**": 2^{13} offsets × 8 bytes = 2^{16}, which exactly covers a 65 535-byte datagram.

Night-Before-the-Exam Sheet

Fact sheet:

DV: $D_x(y) = \min\{c(x,v) + D_v(y)\}$

LS: flood the LSA, then Dijkstra $O(N^2)$

RIP: hops, max 15, 16 = ∞, 30 s, UDP 520

OSPF: IP protocol 89, area 0, AD 110

BGP: path vector, TCP 179, AS_PATH

AD: static 1, eBGP 20, OSPF 110, RIP 120

Hierarchy: ln N levels, e·ln N entries

Token: $S = C/(M - \rho) \cdot \max = C + \rho t$

Circuit: s + x/b + kd

Packet: x/b + (k − 1)p/b + kd

ATM 53 = 5 + 48 · VPI/VCI

T1 1.544 · E1 2.048 · STM-1 155.52

MPLS: label 20 · TC 3 · S 1 · TTL 8

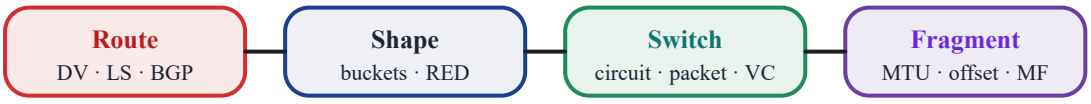
offset = byte position / 8 (13 bits)

Top 14 one-liners for MCQs

1. Router = layer 3, bridge/switch = 2, hub/repeater = 1
2. Distance vector uses Bellman-Ford
3. Link state uses Dijkstra
4. BGP is the inter-AS (EGP) protocol
5. Split horizon reduces count-to-infinity
6. All best routes to one destination = a sink tree
7. Flooding tries every path: robust, many duplicates
8. Leaky bucket → steady output rate
9. Token bucket → bursts pass, tokens are saved
10. RED watches the average queue and drops early, at random
11. X.25 = hop-by-hop error control
12. Frame Relay uses DLCI; ATM uses VPI/VCI
13. IPv4 reassembles only at the destination
14. IPv6 routers never fragment

Top traps: metric ≠ administrative distance · split horizon fails in a loop of 3+ routers · a switch divides the collision domain, not the broadcast domain · virtual circuit = packet switching · leaky bucket drops packets, token bucket drops tokens · offsets count in 8-byte units, and every fragment's data must be a multiple of 8 · ATM's HEC covers the header only

All tricks together: RIP counts hops, OSPF cost, BGP ASes · OSPF "89 · 110 · area 0 · 5/6 · 10/40" · RPF = did it arrive along the way home? · leaky = a tap, token = a savings account · X.25 checks every hop, FR nowhere, ATM cuts to 53 · offset × 8 = byte position



the four big questions of the network layer

✓ Practice MCQs (exam pattern, 10 marks)

1. Distance-vector routing is based on which algorithm?
   (a) Dijkstra (b) Prim
   (c) Bellman-Ford (d) Kruskal

2. In RIP a hop count of 16 means:
   (a) unreachable (b) the best route
   (c) 16 routers (d) the default route

3. OSPF runs straight over IP in protocol number:

(a) 6 (b) 17

(c) 520 (d) 89

4. Which protocol routes between autonomous systems?

(a) BGP (b) RIP

(c) OSPF (d) IS-IS

5. The leaky bucket algorithm mainly gives:

(a) the shortest path (b) a steady output rate

(c) error detection (d) address resolution

6. Token bucket: C = 250 KB, ρ = 2 MB/s, M = 25 MB/s. Burst length ≈

(a) 125 ms (b) 10 ms

(c) 11 ms (d) 23 ms

7. Which device divides the broadcast domain?

(a) hub (b) router

(c) switch (d) repeater

8. The size of an ATM cell:

(a) 48 bytes (b) 64 bytes

(c) 1500 bytes (d) 53 bytes

9. 4000-byte datagram (20 B header), MTU 1500: offset of the last fragment?

(a) 370 (b) 2960

(c) 185 (d) 3

10. Hop-by-hop error control is a feature of:

(a) Frame Relay (b) X.25

(c) ATM (d) MPLS

Answer key → 1-c · 2-a · 3-d · 4-a · 5-b · 6-c · 7-b · 8-d · 9-a · 10-b

Written question: Explain, with an example, the count-to-infinity problem of distance-vector routing, and describe two techniques that limit it. 8 marks

1. A DV router only knows the distances held by its neighbours (Bellman-Ford), so it cannot tell whether the neighbour's route runs back through itself 1
2. Chain A–B–C, each cost 1. B: A = 1; C: A = 2 (via B). The A–B link then fails 1
3. B hears C say "A = 2", so it takes A = 3 via C; then C takes A = 4 via B; then 5, 6, … counting up to ∞ (16 in RIP), with a routing loop running throughout 3
4. **Split horizon**: C does not advertise its A route back to B, the interface it learned it on; **poison reverse**: C advertises A = ∞ to B 2
5. There are also the hold-down timer and triggered updates; none of them fully cures a loop of 3+ routers, which is why link state is preferred 1

Before the exam ✓: Do a Dijkstra table on a 5-node graph · Do a DV update with Bellman-Ford · Do S = C/(M − ρ) and C + ρt · Do fragment sizes, offsets and MF · Write out more practice: DV vs LS, leaky vs token bucket, X.25 vs FR vs ATM